

1. Project Overview

Resume Job Matcher is a web application that helps job seekers find the best matching job postings from a dataset of 680 Indeed job listings. The user uploads their resume as a PDF, and the system uses Natural Language Processing (NLP) to compare the resume against all job descriptions and returns the top matches ranked by relevance.

The project has two architecture generations. The old version used Flask + spaCy + Apache Tika, which was broken and hard to maintain. The current working version uses Streamlit for the frontend and FastAPI for the backend, with scikit-learn's TF-IDF vectorizer for text matching.

What I Built

- A Streamlit web app where users upload their PDF resume
- A FastAPI backend that processes resume text and finds matching jobs
- A TF-IDF + Cosine Similarity matching engine for accurate job matching
- Data scraping pipeline to collect 680 job postings from Indeed Indonesia

Technologies Used

- Frontend: Streamlit (Python-based interactive web app)
- Backend: FastAPI (modern, fast Python REST API framework)
- Matching Engine: scikit-learn (TF-IDF Vectorizer + Cosine Similarity)
- PDF Text Extraction: pdfplumber (replaces old Java-based Tika)
- Data Handling: pandas for CSV/dataframe operations
- Visualization: matplotlib for match score bar charts
- Dataset: 680 job postings scraped from Indeed Indonesia

2. Project File Structure

The project contains both old (legacy) and new (active) files. Only the app/ and backend/ folders are actively used.

Active Files (Currently Used)

app/app.py	-> Streamlit frontend (UI, PDF upload, results display)
backend/main.py	-> FastAPI server with /match and /health endpoints
backend/matching.py	-> Core matching engine (TF-IDF + cosine similarity)
indeed_data.csv	-> Dataset of 680 job postings from Indeed

Legacy Files (Not Used, Kept for Reference)

app.py	-> Old Flask app (replaced by Streamlit)
resume_screening/	-> Old matching modules (broken, replaced by backend/)
templates/	-> Old HTML templates for Flask
static/	-> Old CSS, JS, images for Flask frontend

Configuration Files

- requirements.txt - Lists all Python dependencies
- Procfile - Heroku deployment config
- runtime.txt - Python 3.9.12 for Heroku
- nltk.txt - NLTK data packages needed on Heroku

Dataset Structure (indeed_data.csv)

The CSV file has 680 rows and 5 columns:

title	-> Job title (e.g., 'Data Scientist')
company	-> Company name (e.g., 'Grab', 'GO-JEK')
salary	-> Salary info (mostly 'None')
description	-> Full job description text
link	-> Indeed URL to the original posting

3. How the System Works (End-to-End Flow)

Step 1: Server Startup

When FastAPI starts, it creates a JobMatcher instance. This loads indeed_data.csv, drops rows with missing descriptions, deduplicates by title and company, cleans all job descriptions using the cleaningText() function, then fits a single TF-IDF vectorizer on all cleaned descriptions. The resulting TF-IDF matrix is stored in memory for fast matching.

Step 2: User Uploads Resume

The user opens the Streamlit app in their browser. They upload their resume as a PDF file using the file uploader. The app uses pdfplumber to extract raw text from all pages of the PDF.

Step 3: Send to Backend

When the user clicks 'Find Matching Jobs', the Streamlit app sends an HTTP POST request to http://localhost:8001/match with a JSON payload containing the resume text and the number of top results to return (default 20).

Step 4: Backend Processes Request

The /match endpoint passes the resume text to JobMatcher.match_resume(), which: (a) cleans the resume text using the same cleaningText() function used for jobs, (b) transforms it using the pre-fitted TF-IDF vectorizer, (c) computes cosine similarity between the resume vector and all 680 job vectors, (d) sorts by similarity score descending, (e) returns the top 20 results with title, company, salary, match percentage, and link.

Step 5: Display Results

Streamlit receives the JSON response and displays the results in two ways: a table showing Job Title, Company, Salary, Match %, and Link; and a horizontal bar chart using matplotlib showing the match scores visually.

4. Technical Details (Interview Questions)

How Does the Matching Algorithm Work?

The matching uses TF-IDF (Term Frequency-Inverse Document Frequency) vectorization. TF-IDF converts text into numerical vectors where each word's importance is weighted by how unique it is across all documents. Common words like 'the', 'and' get low weights while domain-specific terms like 'python', 'machine learning' get high weights. Cosine similarity then measures the angle between the resume vector and each job vector. A higher cosine similarity means a better match.

What is `cleaningText()`?

`cleaningText()` is a text normalization function that preprocesses raw text before vectorization. It removes URLs, hashtags, mentions, numbers, non-ASCII characters, punctuation, extra whitespace, and converts everything to lowercase. This ensures the matching focuses on meaningful words rather than formatting differences.

Why TF-IDF Over the Old `CountVectorizer`?

The old system created a new `CountVectorizer` for every single job row, which was extremely slow. The new system fits one `TfidfVectorizer` on all jobs at startup and uses `transform()` for new resumes. TF-IDF also weights words by importance, while simple counting treats all words equally.

Frontend-Backend Communication

The Streamlit frontend and FastAPI backend communicate via HTTP REST calls. Streamlit sends a POST request with JSON payload to the `/match` endpoint. FastAPI processes the request and returns a JSON response. This decoupled architecture means the frontend and backend can be developed and deployed independently.

API Endpoints

POST `/match`

Request: `{"resume_text": "...", "top_n": 20}`

Response: `{"results": [...], "total_jobs": 680}`

Purpose: Takes resume text, returns ranked job matches

GET `/health`

Response: `{"status": "ok", "jobs_loaded": 680}`

Purpose: Health check to confirm server is running

5. Summary & Interview Talking Points

What Problem Does This Solve?

Job seekers spend hours manually searching through job boards. This system automatically ranks job postings based on how well they match a candidate's resume, saving time and helping candidates focus on the most relevant opportunities.

Key Technical Decisions

- FastAPI over Flask: Modern, faster, automatic API docs, async support
- Streamlit over custom HTML/CSS: Rapid prototyping, built-in widgets
- TF-IDF over CountVectorizer: Better accuracy, single fit, faster inference
- pdfplumber over Tika: No Java dependency, pure Python, lighter weight
- CSV over Database: Simple, sufficient for 680 jobs, no setup needed

System Architecture (Simple Words)

Think of it like this: The user gives their resume to the Streamlit app (front desk). The Streamlit app sends it to the FastAPI backend (back office). The backend uses math (TF-IDF and cosine similarity) to compare the resume against 680 job descriptions. It finds the top 20 closest matches and sends them back. The frontend displays them as a table and a chart.

What I Learned

- Building REST APIs with FastAPI and Pydantic validation
- Text preprocessing and NLP techniques for document matching
- TF-IDF vectorization and cosine similarity for similarity search
- Connecting frontend and backend via HTTP requests
- PDF text extraction using pdfplumber
- Data visualization with matplotlib

Possible Improvements

- Add user authentication and saved search history
- Use embeddings (BERT/sentence-transformers) for deeper semantic matching
- Add skill extraction to show which skills match each job
- Deploy to cloud (AWS/GCP) with a proper database
- Add job alert notifications via email